Vel Tech
High Tech
Dr.Rangarajan Dr.Sakunthala
Engineering College
Autonomous - Affiliated to Anna University
Avadi, Chennai

MICRO-LEVEL PROJECT REPORT

25ML35T / FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

AI-BASED STUDY PLANNER:
A CONSTRAINT-BASED ADAPTIVE TIMETABLER

SUBMITTED BY: ArivumathiG

REGISTER NUMBER: 113025148006

YEAR / SECTION: II/A

SUBMITTED TO: DR. MANOJ KUMAR D S

ASSOCIATE PROFESSOR, DEPARTMENT OF CSE (AI&ML)

ACADEMIC YEAR: 2026–2027

SUBMISSION DATE: 13-10-2026

1. ABSTRACT AND PROJECT OBJECTIVES

1.1 ABSTRACT

THE AI-BASED STUDY PLANNER CREATES A WEEKLY TIMETABLE FROM SUBJECTS, DEADLINES, DIFFICULTY, IMPORTANCE, AND AVAILABLE daily hours. It uses a utility score to prioritise topics and CSP methods to find a timetable that follows scheduling rules. Backtracking, MRV, LCV, and AC-3 support the search. A small syllabus ontology orders prerequisites, while progress tracking allows the planner to revise the schedule after missed sessions.

1.2 Keywords

Rational Agent, Utility Function, CSP, Backtracking, MRV, AC-3, Ontology.

1.3 Problem Statement

Students have limited time to prepare for subjects with different deadlines and difficulty levels. Fixed timetables may ignore prerequisites and missed sessions. This project generates an explainable timetable that respects study-time and subject constraints.

1.4 Objectives

- Prioritise urgent, important, and difficult subjects.
- Generate a timetable within daily-hour limits.
- Schedule prerequisites before dependent topics.
- Compare plain backtracking with MRV + AC-3.
- Re-plan after missed sessions and provide fair subject coverage.

1.5 Scope

The project covers timetable generation, constraint checking, simple syllabus rules, explanations, and re-planning.

2. INTRODUCTION AND PROPOSED SOLUTION

2.1 Background

Study planning is a decision problem because multiple subjects compete for limited time. An AI agent can use inputs and rules to choose what to study next.

2.2 Existing Approach
Students commonly prepare schedules manually or divide time equally among subjects.

2.3 Limitations
- Manual plans may overlook urgent exams or difficult topics.
- Fixed schedules may not recover well when sessions are missed.

2.4 Proposed Solution
The planner scores subjects, assigns study sessions to day/time slots, checks constraints, and displays a timetable with reasons. When a session is missed, it updates progress and generates a revised plan.

2.5 Users and Benefits
Students can use the planner to organise revision, prioritise deadlines, follow prerequisites, and recover from missed sessions

3. AI CONCEPTS AND PROBLEM FORMULATION

3.1 AI Concepts
Utility-based rational agent, Constraint Satisfaction Problem (CSP), heuristic search, rule-based reasoning, and ontology.

3.2 Suitability
Utility scoring ranks study needs; CSP checks timetable validity; heuristics reduce search; ontology and rules support prerequisite order and extra study sessions.

3.3 Problem Formulation

Element	Description
Initial state	Subjects, topics, deadlines, difficulty, weight, daily hours, scores, and progress.
Actions	Assign a session to a day and time slot, or revise it.

Goal	Cover priority topics before exams with all constraints satisfied.

3.4 PEAS

Component	Description
Performance	Priority coverage, deadline compliance, fairness, and valid constraints.
Environment	Student syllabus, subjects, calendar, and available time.
Actuators	Display the timetable and explanations.
Inputs	Deadlines, difficulty, weights, hours, scores, and completed/missed sessions.

3.5 Inputs and Outputs

Inputs include subject details, topics, deadlines, daily hours, prerequisites, scores, and progress. Outputs include a weekly timetable, priority scores, explanations, and revised plans.

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 Architecture Diagram

Student Inputs → Utility Scorer → Ontology and Rules → CSP Scheduler → Constraint Validator → Timetable and Explanations → Progress Update / Re-planning

4.3 Modules

Module	Responsibility
Input manager	Collect and validate inputs.

Utility scorer	Rank topics by urgency, weight, difficulty, and allocated hours.
CSP scheduler	Assign sessions and enforce constraints.
Progress manager	Record completed/missed sessions and re-plan.

4.4 Tools

Python, VS Code or Jupyter Notebook; optionally pandas for tabular data.

5. ALGORITHM AND METHODOLOGY

5.1 Methodology

The planner ranks subjects using utility, creates a variable for each study session, and assigns available day-slot choices. Constraint checks remove invalid choices. Search continues until a valid schedule is found or the constraints cannot all be satisfied.

5.2 Utility Rule

Utility(s) = 0.40 × UrgencyScore + 0.30 × PriorityWeight + 0.20 × DifficultyLoad − 0.10 × HoursAlreadyAllocated

Urgency rises as the exam approaches. Scores should use a common scale. The highest-utility eligible topic is considered first, while hard constraints must still be obeyed.

5.3 CSP Algorithm

1. Create one variable for each required study session.
2. Set each domain to possible day/time slots.
3. Run AC-3 to remove inconsistent values.
4. Choose the variable with the fewest legal values (MRV).
5. Try the least disruptive value first (LCV).
6. Backtrack when a conflict occurs; continue until a valid plan is found.

5.4 Ontology and Rules

Ontology: Subject → Unit → Topic. The relation prerequisiteOf(A, B) means A must be studied before B.

- If Topic A is a prerequisite of Topic B, schedule A first.
- If Difficulty(subject, High) and LastScore(subject) < 50, add two extra sessions.
- If a session is missed, mark it incomplete and assign it to a valid free slot.
- Fairness rule: give every active subject some planned time before repeatedly adding extra time to one subject, unless deadlines require otherwise.

Figure 3. Ontology: Subject → Unit → Topic; Topic A —prerequisiteOf→ Topic B.

5.5 Constraints

- Do not exceed available study hours per day.
- Finish required sessions before the relevant exam.
- Do not place hard-subject sessions back-to-back.
- Keep a minimum gap between revision sessions for the same topic.
- Schedule prerequisites before dependent topics.
- Do not assign two sessions to the same time slot.

5.6 Search Comparison and Uncertainty

Plain backtracking may try many assignments. MRV, LCV, and AC-3 aim to reduce unnecessary search. Measure assignments tried and runtime on the same input; do not invent results. A simple completion estimate is completed sessions divided by planned sessions. After a missed session, update progress and run the scheduler again. This ratio is a rough estimate, not a calibrated probability.

6. IMPLEMENTATION AND GITHUB

6.1 Environment

Python using VS Code or Jupyter Notebook on Windows.

6.2 Implementation Steps

1. Create subject and topic data structures.
2. Implement utility scoring.
3. Represent sessions, domains, and constraints.
4. Implement backtracking, then MRV, LCV, and AC-3.
5. Add prerequisite and extra-session rules.
6. Display the timetable and explanations.
7. progress tracking and re-planning.

6.3

6.3 Execution

Install Python, open the project in VS Code, run `python main.py`, enter the subject details, and review the timetable. Adjust these steps to match the actual code.

6. 4 Commit History

Use meaningful commits such as `add utility scoring`, `implement CSP scheduling`, and `add missed-session replanning`. Report only commits actually made.

7. RESULTS AND OBSERVATIONS

7.1 Example Input and Timetable

Example subjects from the existing report: Data Structures (deadline in 3 days, weight 5, high difficulty), Operating Systems (deadline in 5 days, weight 4, medium difficulty), and Economics (deadline in 10 days, weight 3, low difficulty). Available time: 4 hours/day.

Day	Example allocation
Day 1	Data Structures 3 h; Operating Systems 1 h

Day 2	Data Structures 2 h; Operating Systems 2 h
Day 3	Operating Systems 2 h; Economics 2 h
Day 4	Economics 2 h; Operating Systems 2 h

This is an illustrative priority-based plan. The final CSP code must validate all constraints before calling a timetable valid.

7.2 Test Cases

Test	Scenario	Expected result
1	One subject with enough time	All required sessions scheduled.
2	Subjects have different deadlines	Urgent subject is prioritised when feasible.
3	Daily hours are limited	Daily cap is not exceeded.
4	Topic has a prerequisite	Prerequisite is scheduled first.
5	A session is missed	Progress is updated and timetable is revised.

7.4 Observations

Record constraint satisfaction, deadline coverage, assignments tried, and runtime from actual program runs. Do not report unmeasured performance values.

8. DISCUSSION, LIMITATIONS AND FUTURE WORK

8.1 Discussion

Utility explains why topics are prioritised; CSP checks timetable validity; ontology orders prerequisites; progress tracking supports adaptation.

8.2 Limitations

- The plan depends on accurate input and study-hour estimates.
- Completion ratio is only a rough estimate of finishing on time.
- Large schedules may require more search time.
- The planner cannot automatically know fatigue or interruptions unless recorded.

8.3 Future Enhancements

- Add a graphical interface and calendar export.
- Improve completion estimates using past progress.
- Add flexible breaks and revision spacing.
- Allow users to compare alternative schedules.

8.4 Applications

Useful for school, college, and competitive-exam students.

8.5 Learning Outcomes

The project demonstrates rational-agent utility, CSP, heuristic search, rules, ontology, and adaptive planning.

9. CONCLUSION AND REFERENCES

9.1 Conclusion

The AI-Based Study Planner combines utility-based prioritisation with constraint-based scheduling. It aims to produce an explainable timetable that respects deadlines, daily limits, prerequisites, and revision gaps. MRV, LCV, and AC-3 support efficient search, while progress updates help recover from missed sessions. Final claims should be supported by actual program results.

9.2 References

- Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach.
- Python documentation: https://docs.python.org/3/

- Provided Foundations of AI micro-project report template.